TECHNICAL UNIVERSITY OF MOLDOVA
Faculty of Computers, Informatics, and Microelectronics
Department of Software Engineering
Software Engineering Study Programme

LABORATORY REPORT NO. 3

BEHAVIORAL PATTERNS IN A SHARED TASK-REPORT PROJECT

Student: Lucian-Adrian Gavril

Group: FAF-241

Coordinator: asist. univ., Daniel Istrati

Chișinău, 2026

TABLE OF CONTENTS

ABSTRACT 3

INTRODUCTION 4

1 SELECTED PATTERNS AND IMPLEMENTATION 5

1.1 Command: represent a request as an object 5

1.2 Observer: notify subscribers after a state change 5

1.3 Strategy: choose a task ordering 5

1.4 Reuse of the Lab 1 patterns 5

2 SOURCE-LEVEL TRACE AND RESULTS 6

2.1 Trace of task 2 6

2.2 Build and run 6

2.3 Questions for oral review 6

CONCLUSIONS 8

REFERENCES 9

APPENDICES 10

APPENDIX A – Source files 10

A.1 Single client 10

A.2 Shared task model 12

A.3 Behavioral types 13

ABSTRACT

This report explains the Command, Observer, and Strategy patterns in the behavioral continuation of a task-reporting project. The Lab 3 mode uses the shared `TaskItem` model and the Builder, Factory Method, and Abstract Factory types implemented for Lab 1. A command carries a request to complete a task; the task board changes the shared model and notifies an observer; a sorting strategy orders the remaining work. The report traces this flow through the single command-line client, gives the expected output, and includes commands for building and running both laboratory modes.

Keywords: behavioral patterns, command, observer, strategy, task board

INTRODUCTION

The TMPS Laboratory 3 brief asks for three behavioral patterns and familiarity with the others. The linked laboratory instructions describe this work as a continuation of the previous project, require the same repository, and specify one client for the whole system [1, 2]. This implementation keeps one executable project, one `Program.Main`, and one shared `TaskItem` model.

The `lab-03` command-line mode demonstrates Command, Observer, and Strategy in the same task-reporting domain used by Lab 1. It also uses the Lab 1 builder to configure output, the Factory Method creator to produce a CSV report, and the Abstract Factory renderer to produce a matching Markdown board. The following sections trace the concrete path rather than treating these patterns as isolated examples.

1 SELECTED PATTERNS AND IMPLEMENTATION

This chapter maps the three behavioral patterns to `BehavioralPatterns.cs`. The single client is in `../lab-01/Program.cs`; the shared model is in `../lab-01/TaskItem.cs`. The Lab 1 creational types are in `../lab-01/CreationalPatterns.cs`.

1.1 Command: represent a request as an object

`ITaskCommand` defines an `Execute(TaskBoard)` operation. `CompleteTaskCommand` stores the task identifier in its constructor and calls `TaskBoard.CompleteTask`. In `Program.RunLab03`, the client constructs `CompleteTaskCommand(2)` and passes it to `TaskBoard.Execute`. The command carries the requested action; the board owns the state change.

The request object keeps its data and operation together. The current code does not add an undo stack or queue, so those remain possible extensions rather than existing features.

1.2 Observer: notify subscribers after a state change

The `ITaskObserver` interface defines the completion callback. `ConsoleTaskObserver` prints the event. `Program.RunLab03` subscribes the observer before executing the command. The board marks the shared task complete and then calls `NotifyObservers`, so the subscriber receives the updated task.

The board depends on the observer interface rather than a particular output action. The example does not implement unsubscribe behavior or prevent duplicate subscriptions.

1.3 Strategy: choose a task ordering

The `ITaskSortStrategy` interface defines the sorting operation. `PrioritySortStrategy` orders by descending priority and then by due date. `DueDateSortStrategy` orders by due date and then by descending priority. The Lab 3 client supplies `PrioritySortStrategy` to `TaskBoard.GetTasks`; the board delegates ordering through the interface.

1.4 Reuse of the Lab 1 patterns

The same client constructs `TaskReportOptionsBuilder` to set the title and minimum priority. It passes the selected tasks to `CsvTaskReportCreator`, whose Factory Method creates the formatter. After the command changes task 2, `TaskBoardRenderer` uses `MarkdownBoardThemeFactory` to render the remaining tasks with a matching Markdown heading and rows. The behavioral work therefore extends the existing report project and its shared model.

2 SOURCE-LEVEL TRACE AND RESULTS

This chapter follows the data and calls in `Program.RunLab03`. The trace is taken from the executable source; it is not a timing or performance measurement.

2.1 Trace of task 2

The client creates three open tasks and builds options titled `Study tasks` with minimum priority 1. The priority strategy orders them as 1, 2, 3. The CSV report is printed before the completion request. The client then executes `CompleteTaskCommand(2)`. The board marks task 2 complete and notifies the console observer. The client sorts and filters again, so task 2 is omitted from the final open-task board.

Table 2.1 – Shared tasks before and after the behavioral command

ID	Task	Priority	Due date	After command
1	Write report	3	12 Oct 2026	Open
2	Review notes	2	11 Oct 2026	Complete
3	Archive old draft	1	13 Oct 2026	Open

The expected output is:

```
Study tasks
Id,Title,Priority,Status
1,"Write report",3,Open
2,"Review notes",2,Open
3,"Archive old draft",1,Open
Observer: task 2 was completed.
# Study tasks
- [ ] Write report (P3)
- [ ] Archive old draft (P1)
```

2.2 Build and run

From the `lab-03` directory, build the shared executable and run its behavioral mode with the .NET 10 SDK:

```
$build = Join-Path $env:TEMP 'tmps-build'
dotnet build ..\lab-01\TMPSLab01.csproj -c Release --artifacts-path $build
dotnet (Join-Path $build 'bin\TMPSLab01\release\TMPSLab01.dll') lab-03
```

The same project runs the creational-pattern demonstration with the `lab-01` argument. Both modes use the same task model and one application entry point.

2.3 Questions for oral review

1. What does `CompleteTaskCommand` store, and which object changes the task? It stores the task ID; `TaskBoard` changes the shared task.
2. Why does the board notify observers after `MarkComplete`? Subscribers receive the task after its state has changed.

3. How could a second observer be added? Implement `ITaskObserver` and subscribe it; the board's notification loop need not change.
4. How can the client use due-date order? Pass the due-date strategy to the board.
5. How does Strategy differ from State? The caller chooses an algorithm here; State changes behavior in response to the object's lifecycle state.
6. What does Lab 3 reuse from Lab 1? The task model, options builder, CSV report creator, and Markdown board renderer.

CONCLUSIONS

Command packages the completion request, the board changes the shared task and notifies its observer, and Strategy supplies the ordering rule. The Lab 3 mode also reuses the Builder, Factory Method, and Abstract Factory implementations from Lab 1. The two modes run through one client in one executable project, and the behavioral continuation uses the same task model.

REFERENCES

1. Technical University of Moldova, ELSE. *FAF.TMPS21.1: Laboratory work #3*, Behavioral Design Patterns. Available online: `https://else.fcim.utm.md/mod/assign/view.php?id=27829` (accessed 10 October 2026).

2. V. Drumea. *SDTM-Labs, Lab #3 README*. Continuation, same-repository, one-client, documentation, and source-grouping requirements. Available online: `https://github.com/DrVasile/SDTM-Labs/blob/master/Lab%233/README.md` (accessed 10 October 2026).

3. E. Gamma, R. Helm, R. Johnson, and J. Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994. The course lists this work in its references section.

APPENDICES

APPENDIX A – Source files

The following listings show the single client, shared model, and behavioral types used in the trace above. The creational types reused by the Lab 3 mode are listed in the Laboratory 1 report.

A.1 Single client

Listing A.1: Single command-line client for both laboratories

```
using System;
using System.Collections.Generic;
using System.Linq;
using TMPS.Behavioral;
using TMPS.Creational;
using TMPS.Domain;

namespace TMPS.Client;

internal static class Program
{
    private static int Main(string[] args)
    {
        if (args.Length == 0 || args[0] == "lab-01")
        {
            RunLab01();
            return 0;
        }

        if (args.Length == 1 && args[0] == "lab-03")
        {
            RunLab03();
            return 0;
        }

        Console.Error.WriteLine("Usage: TMPSLab01 [lab-01|lab-03]");
        return 1;
    }

    private static void RunLab01()
    {
        List<TaskItem> allTasks = new()
```

```
    {
        new TaskItem(1, "Write report", 3, new DateOnly(2026, 10, 12), false),
        new TaskItem(2, "Review notes", 2, new DateOnly(2026, 10, 11), false),
        new TaskItem(3, "Archive old draft", 1, new DateOnly(2026, 10, 10), true)
    };

    TaskReportOptions options = new TaskReportOptionsBuilder()
        .WithTitle("Study tasks")
        .WithMinimumPriority(2)
        .Build();

    List<TaskItem> selectedTasks = SelectTasks(allTasks, options);

    TaskReportCreator reportCreator = new CsvTaskReportCreator();
    Console.WriteLine(reportCreator.Build(options.Title, selectedTasks));

    TaskBoardRenderer board = new(new MarkdownBoardThemeFactory());
    Console.WriteLine(board.Render(options.Title, selectedTasks));
}

private static void RunLab03()
{
    TaskBoard board = new(new List<TaskItem>
    {
        new(1, "Write report", 3, new DateOnly(2026, 10, 12), false),
        new(2, "Review notes", 2, new DateOnly(2026, 10, 11), false),
        new(3, "Archive old draft", 1, new DateOnly(2026, 10, 13), false)
    });

    TaskReportOptions options = new TaskReportOptionsBuilder()
        .WithTitle("Study tasks")
        .WithMinimumPriority(1)
        .Build();
    ITaskSortStrategy sortStrategy = new PrioritySortStrategy();

    board.Subscribe(new ConsoleTaskObserver());

    List<TaskItem> tasksBeforeCommand = SelectTasks(board.GetTasks(sortStrategy),
options);
    TaskReportCreator reportCreator = new CsvTaskReportCreator();
    Console.WriteLine(reportCreator.Build(options.Title, tasksBeforeCommand));
```

```
        board.Execute(new CompleteTaskCommand(2));

        List<TaskItem> openTasks = SelectTasks(board.GetTasks(sortStrategy), options)
    ;
        TaskBoardRenderer boardRenderer = new(new MarkdownBoardThemeFactory());
        Console.WriteLine(boardRenderer.Render(options.Title, openTasks));
    }

    private static List<TaskItem> SelectTasks(
        IEnumerable<TaskItem> tasks,
        TaskReportOptions options)
    {
        return tasks
            .Where(task => options.IncludeCompleted || !task.IsCompleted)
            .Where(task => task.Priority >= options.MinimumPriority)
            .ToList();
    }
}
```

A.2 Shared task model

Listing A.2: Task model shared by both laboratories

```
using System;

namespace TMPS.Domain;

internal sealed class TaskItem
{
    public TaskItem(int id, string title, int priority, DateOnly dueDate, bool
    isCompleted)
    {
        Id = id;
        Title = title;
        Priority = priority;
        DueDate = dueDate;
        IsCompleted = isCompleted;
    }

    public int Id { get; }
    public string Title { get; }
    public int Priority { get; }
    public DateOnly DueDate { get; }
```

```
    public bool IsCompleted { get; private set; }

    public void MarkComplete()
    {
        IsCompleted = true;
    }
}
```

A.3 Behavioral types

Listing A.3: Command, Observer, Strategy, and task board

```
using System;
using System.Collections.Generic;
using System.Linq;
using TMPS.Domain;

namespace TMPS.Behavioral;

internal interface ITaskCommand
{
    void Execute(TaskBoard board);
}

internal sealed class CompleteTaskCommand : ITaskCommand
{
    private readonly int _taskId;

    public CompleteTaskCommand(int taskId)
    {
        _taskId = taskId;
    }

    public void Execute(TaskBoard board)
    {
        board.CompleteTask(_taskId);
    }
}

internal interface ITaskObserver
{
    void OnTaskCompleted(TaskItem task);
}
```

```
internal sealed class ConsoleTaskObserver : ITaskObserver
{
    public void OnTaskCompleted(TaskItem task)
    {
        Console.WriteLine($"Observer: task {task.Id} was completed.");
    }
}

internal sealed class TaskBoard
{
    private readonly List<TaskItem> _tasks;
    private readonly List<ITaskObserver> _observers = new();

    public TaskBoard(IEnumerable<TaskItem> tasks)
    {
        _tasks = tasks.ToList();
    }

    public void Subscribe(ITaskObserver observer)
    {
        _observers.Add(observer);
    }

    public void Execute(ITaskCommand command)
    {
        command.Execute(this);
    }

    public void CompleteTask(int taskId)
    {
        foreach (TaskItem task in _tasks)
        {
            if (task.Id == taskId)
            {
                task.MarkComplete();
                NotifyObservers(task);
                return;
            }
        }

        throw new InvalidOperationException($"Task {taskId} was not found.");
```

```
    }

    public IEnumerable<TaskItem> GetTasks(ITaskSortStrategy strategy)
    {
        return strategy.Sort(_tasks);
    }

    private void NotifyObservers(TaskItem task)
    {
        foreach (ITaskObserver observer in _observers)
        {
            observer.OnTaskCompleted(task);
        }
    }
}

internal interface ITaskSortStrategy
{
    IEnumerable<TaskItem> Sort(IEnumerable<TaskItem> tasks);
}

internal sealed class PrioritySortStrategy : ITaskSortStrategy
{
    public IEnumerable<TaskItem> Sort(IEnumerable<TaskItem> tasks)
    {
        return tasks
            .OrderByDescending(task => task.Priority)
            .ThenBy(task => task.DueDate);
    }
}

internal sealed class DueDateSortStrategy : ITaskSortStrategy
{
    public IEnumerable<TaskItem> Sort(IEnumerable<TaskItem> tasks)
    {
        return tasks
            .OrderBy(task => task.DueDate)
            .ThenByDescending(task => task.Priority);
    }
}
```